Trusted Code Generation:

A Formal Trust-Boundary Framework for Specification-Conditioned Smart-Contract Synthesis and Verification

June 27, 2026

Abstract

Large language models (LLMs) can generate smart-contract code from informal human intent, but their outputs carry no formal guarantee: a fluent generation is statistically plausible, not provably correct. We present *Trusted Code Generation* (TCG), a framework that recovers a meaningful correctness guarantee not by trusting the generative model but by isolating it. TCG draws an explicit trust boundary between an *untrusted* generation stage—which may produce arbitrary or adversarial output—and a *trusted, formal* verification core built on a sound formal semantics of the Ethereum Virtual Machine (EVM). We separate three layers that an informal treatment tends to conflate: the operational checker that consumes a certificate, the semantic truth of a claim over the formal EVM semantics, and the soundness assumption that bridges them. With this separation the central soundness theorem becomes a genuine result rather than a restatement of a definition. The design claim is that system soundness is *invariant to the capabilities of the generative model*: any accepted artifact is verified against an EVM-level specification, and non-accepted artifacts are conservatively rejected. We enrich the specification beyond a terminal postcondition to a claim over full machine configurations, including trace events and exit status, so that properties such as reentrancy, revert behavior, and emitted logs fall within the guarantee. We isolate the single *non-formal* gap—the alignment between the formal specification and the human's intent—and discharge it as an explicit human-oracle assumption rather than concealing it, while keeping the human outside the formal core. A completeness corollary identifies unspecified behavior as the system's blind spot and motivates specification completeness as the principal direction for future work.

1 Introduction

The translation of informal human intent into executable code is now routinely performed by large language models. In the smart-contract setting this is attractive and dangerous in equal measure: contracts are short, their execution environment is deterministic, and they custody real value, so an undetected defect is directly monetizable by an adversary. An LLM that emits Solidity from a natural-language description offers speed, but the artifact it produces is *plausible*, not *proven*. In the vocabulary of epistemology, the model supplies neither justification nor a guarantee of truth; it supplies a high-likelihood sample from a learned distribution over token sequences.

A natural reaction is to make the generator "more reliable"—better training, more reasoning, self-critique. We argue this is the wrong axis. No amount of generative competence converts a statistical sample into a proof, because the two belong to different categories. Instead, TCG accepts the generator as permanently untrusted and relocates the guarantee to a component that can actually carry it: a sound verifier operating over a formal semantics of the target machine. The generator

(informally, "vibe coding") is then free to be wrong as often as it likes; its errors are caught, not trusted.

Scope. TCG applies precisely when the domain admits a formal semantics. Smart contracts on the EVM are a near-ideal instance: the EVM is deterministic once its execution environment is fixed (Section 3.1) and its execution is bounded by gas, and a complete executable formal semantics, KEVM [1], exists and is validated against the official Ethereum test suite. Tasks without a formal notion of success—"design a profitable business"—fall outside the framework entirely, not because verification is expensive there but because no correctness predicate exists to verify against. The boundary of applicability is exactly the boundary of formalizability.

Contributions.

1. A three-layer formalization—operational checker, semantic truth, and a bridging soundness assumption—that makes the soundness theorem a genuine result rather than a consequence of how the verifier is defined (Sections 3.4 and 5).
2. A pipeline model assigning an explicit trust status—untrusted (U), trusted-formal (T), or non-formal trusted oracle (T^*)—to every component, with the human kept outside the formal core (Section 3.3).
3. A specification model over full machine configurations—precondition, trace invariant, postcondition, and exit status—so that trace-sensitive security properties are expressible and covered by the theorem (Section 3.2).
4. A completeness corollary that formalizes the specification blind spot and reframes specification completeness, with an explicitly heuristic coverage proxy, as the central open problem (Section 6).

2 Background and Setting

2.1 The barrier to a verified translator

It is tempting to seek a *provably correct* translator from natural language to formal specification, analogous to a verified compiler such as CompCert [2]. This is impossible in principle, and the impossibility is categorical rather than engineering. A compiler can be proven correct because both endpoints carry a formal semantics: one proves $[\![\text{src}]\!] = [\![\text{compile(src)}]\!]$. Natural language has no formal semantics. There is nothing to place on the left-hand side of such an equation, so "correctness of translation" is not a well-defined predicate. The consequence for system design is decisive: the goal cannot be to *trust* the translation; it can only be to make an incorrect translation *detectable*, and to render the residual, undetectable gap explicit. This posture is reinforced by empirical findings in autoformalization, where minor paraphrases of an identical natural-language statement produce materially different formal outputs [4], and where the alignment between an informal statement and its formalization must itself be evaluated rather than assumed [5].

2.2 A semantics-first verification target

We assume a sound, executable formal semantics of the EVM that defines a transition relation over machine configurations, validated for faithfulness against a reference implementation by an

extensive conformance suite [1]. Independent formalizations of EVM semantics in other systems—for example in Dafny [3]—make cross-validation of a critical property feasible and provide leverage against the risk that any single semantics mismodels the machine. Verifying at the level of deployed bytecode—rather than source—removes the source compiler from the trusted base: whatever the compiler emits, the verifier reasons about the actual deployed artifact. This choice determines where the trust boundary can be drawn and is assumed throughout.

3 The TCG Pipeline

3.1 Machine configurations and the environment

The EVM is deterministic only once its execution environment is fixed. We therefore take states to be full configurations rather than abstract points.

Definition 1 (Configurations)**.** Let Σ be the set of full EVM/world-state configurations, each including account state, persistent storage, contract code, balances, the transaction parameters (caller, calldata, value), block parameters, remaining gas, the active fork rules and fee schedule, precompile semantics, and the code of any reachable external contract. Let $\mathcal{E}$ denote an *environment class*: a set of configurations agreeing on fork rules, fee schedule, and the deployment assumptions on caller, calldata shape, and external code. The real machine induces a transition relation $\rightarrow_{\mathrm{EVM}}^{f,g}$ parameterized by fork f and fee schedule g; the formal semantics induces $\rightarrow_{K}^{f,g}$ likewise.

A *trace* is a finite sequence of configurations $\sigma_0 \rightarrow \sigma_1 \rightarrow \cdots \rightarrow \sigma_m$ together with the externally observable events emitted along it (logs, external calls, value transfers) and the terminating *exit status* $e \in \{\mathsf{halt}, \mathsf{revert}, \mathsf{oog}, \mathsf{exc}\}$ (successful halt, revert, out-of-gas, exceptional halt).

3.2 Specifications over configurations

To express trace-sensitive properties—reentrancy, callback discipline, event emission, and the success/revert distinction—a specification cannot be a pair of state predicates alone.

Definition 2 (Specification)**.** A specification is a tuple $\varphi = (\mathrm{Pre}, \mathrm{Trace}, \mathrm{Post}, \mathrm{Exit})$, where Pre and Post are predicates over configurations, Trace is a predicate over traces (a trace invariant, e.g. "no external call re-enters this contract before its first invocation returns"), and $\mathrm{Exit} \subseteq \{\mathsf{halt}, \mathsf{revert}, \mathsf{oog}, \mathsf{exc}\}$ constrains the admissible exit statuses. A trace τ from σ_0 *satisfies* φ, written $\tau \models \varphi$, if $\sigma_0 \models \mathrm{Pre}$ implies that $\mathrm{Trace}(\tau)$ holds, the exit status of τ lies in Exit, and the terminal configuration satisfies Post. The partial-correctness terminal claim is the special case $\mathrm{Trace} \equiv \mathsf{true}$, $\mathrm{Exit} = \{\mathsf{halt}\}$.

3.3 Components and trust status

Let $\mathcal{C}$ be the set of proof certificates and $\mathcal{A} = \mathcal{A}_{\mathrm{chk}} \uplus \mathcal{A}_{\mathrm{asm}}$ the set of annotations, partitioned into those the checker fully discharges as obligations ($\mathcal{A}_{\mathrm{chk}}$: loop invariants, ranking functions, predicate strengthenings) and those it would otherwise consume as unverified facts ($\mathcal{A}_{\mathrm{asm}}$: e.g. storage-layout

hints). Components and their trust status:

$F : \mathcal{N} \to \mathcal{S} \cup \{\bot\}$	$[\mathsf{U}]$	formalizer
$G : \mathcal{S} \times \mathrm{CE}^* \to \mathcal{B}$	$[\mathsf{U}]$	generator
$I : \mathcal{S} \times \mathcal{B} \to \mathcal{A} \times \mathcal{C}$	$[\mathsf{U}]$	synthesizer
$\mathrm{Check} : \mathcal{S} \times \mathcal{B} \times \mathcal{A} \times \mathcal{C} \to \mathrm{Res}$	$[\mathsf{T}]$	checker
$H : \mathcal{N} \times \mathcal{S} \to \{\mathrm{ok}, \mathrm{rej}\}$	$[\mathsf{T}^*]$	human oracle

where the checker's result type is structured,

$$\mathrm{Res} \;=\; \mathrm{Accept}(\mathcal{C}) \;\uplus\; \mathrm{Refute}(\mathrm{CE}) \;\uplus\; \mathrm{Unknown},$$

distinguishing a discharged proof, a concrete counterexample trace, and a failure to decide (timeout, solver incompleteness). The marker T^* denotes *non-formal trusted*: H is the sole bearer of the semantics of $\mathcal{N}$, belongs to the trusted base but *not* to the formal core, and cannot be replaced by any formal artifact.

Remark 1 (The two trusted parts are distinct). The *formal trusted core* consists of the checker together with the proof checker and the formal EVM semantics:

$$\text{formal core} \;=\; \{\,\mathrm{Check},\ \text{proof checker},\ \text{formal EVM semantics}\,\}.$$

The *non-formal trusted assumption* is H. The abstract's "trusted formal core" refers only to the former; the human is a trusted *assumption*, not a verification component. Components F, G, I are untrusted and may behave arbitrarily, including adversarially. In particular the certificate and the checked annotations $\mathcal{A}_{\mathrm{chk}}$ are *inputs* to Check, never axioms; the checker does not believe an invariant, it discharges it. Annotations in $\mathcal{A}_{\mathrm{asm}}$, by contrast, are not self-checking and must be either eliminated or promoted to an explicit member of the trusted base (Assumption 5).

3.4 The verification predicate: three layers

We deliberately separate the operational checker from semantic truth, so that the bridge between them is an assumption and not a definition.

Definition 3 (Operational acceptance). $\mathrm{Check}(\varphi, b, a, c) = \mathrm{Accept}(c)$ holds iff the checker mechanically validates the certificate c as discharging every proof obligation that the claim "b satisfies φ over $\to_K$" generates under annotations a. This is a decidable, syntactic procedure on c; it makes no reference to truth.

Definition 4 (Semantic claim). Write $\to_K^{f,g} \models \varphi(b)$ for the semantic statement that, in the formal semantics under fork f and fee schedule g, every terminating $\to_K^{f,g}$-trace of b from a configuration satisfying Pre satisfies φ in the sense of Definition 2 (trace invariant, exit status, and postcondition).

These two are linked only through the soundness of the checker, stated as Assumption 3 below; neither implies the other by definition. The value Unknown and any Refute($\cdot$) are both treated as non-acceptance: an obligation that is not discharged is not accepted. Only Refute(ce) carries a counterexample that can be fed back to the generator; Unknown carries none.

3.5 The process

Definition 5 (TCG). For intent $n \in \mathcal{N}$, the process proceeds stepwise (avoiding any ill-typed application of H to $\bot$):

1. If $F(n) = \bot$, return $\bot_{\text{spec}}$.
2. Otherwise let $\varphi = F(n)$. If $H(n, \varphi) = \text{rej}$, return $\bot_{\text{spec}}$.
3. Otherwise iterate for $k = 1, \ldots, K_{\max}$: let $b_k = G(\varphi, \text{ce}_{<k})$ and $(a_k, c_k) = I(\varphi, b_k)$.
   - If $\text{Check}(\varphi, b_k, a_k, c_k) = \text{Accept}(c_k)$, return (b_k, φ).
   - If the result is $\text{Refute}(\text{ce}_k)$, append ce_k to the feedback and continue.
   - If the result is Unknown, continue with no feedback.
4. If no iteration accepts, return $\bot_{\text{gen}}$.

The process has three outcomes, all safe: abstention at the specification stage ($\bot_{\text{spec}}$), failure to synthesize an accepted artifact within budget ($\bot_{\text{gen}}$), or an accepted pair (b, φ). The system never emits a non-accepted artifact labeled as accepted; this is the soundness of the *construction*, prior to any environmental assumption.

4 Assumptions (Trusted Base)

The guarantee holds relative to an explicit trusted base T.

Assumption 1 (Model adequacy). $\to_K^{f,g}$ agrees with $\to_{\text{EVM}}^{f,g}$ on the target fork f and fee schedule g, over the environment class $\mathcal{E}$. Status: *validated* against the official conformance suite [1], not proved.

Assumption 2 (Environment adequacy). The environment encoded in Σ and $\mathcal{E}$ matches deployment: caller, calldata class, external contract code, precompiles, block parameters, chain id, balances, gas limits, and fork version.

Assumption 3 (Checker soundness). The checker and its proof/SMT backend are sound: if $\text{Check}(\varphi, b, a, c) = \text{Accept}(c)$ then $\to_K^{f,g} \models \varphi(b)$. This is the bridge between Definitions 3 and 4.

Assumption 4 (Artifact identity). The verified bytecode is bit-identical to the deployed runtime bytecode, accounting for constructor versus runtime code and immutables.

Assumption 5 (Annotation discipline). Every annotation used by Check lies in $\mathcal{A}_{\text{chk}}$ and is discharged as an obligation; any $\mathcal{A}_{\text{asm}}$ fact (e.g. storage layout) is either eliminated or listed explicitly in T with an independent justification.

Assumption 6 (Specification well-formedness). Pre is satisfiable and describes reachable deployment configurations, so that the guarantee is non-vacuous.

Assumption 7 (Human validation). $H(n, \varphi) = \text{ok}$, and the human accepts the contract that *the artifact will be verified exactly against* φ: the specification is declared the oracle of correctness. This is a trusted *assumption*, not a formal step.

Assumption 7 is the methodological core. It converts the undecidable claim "φ matches intent" into an explicit, shared premise. We do not assert that $\varphi \approx n$ is provable—it is not. We make the dependence on a human judgment a visible part of the theorem rather than a hidden defect.

5 Soundness

Theorem 1 (Soundness of TCG). *Under Assumptions* 1–7*, if the process returns* (b, φ)*, then for every real EVM trace* τ *of the deployed bytecode* b*, under fork* f *and fee schedule* g*, starting from any reachable configuration satisfying* Pre*, the trace satisfies* φ*: its events obey the trace invariant* Trace*, its exit status lies in* Exit*, and its terminal configuration satisfies* Post*.*

Proof. The process returns (b, φ) only via $\mathrm{Check}(\varphi, b, a, c) = \mathrm{Accept}(c)$ for some checked annotations $a \in \mathcal{A}_{\mathrm{chk}}$ and certificate c (Definition 5, Assumption 5). By Assumption 3, operational acceptance entails the *semantic* claim $\rightarrow_K^{f,g} \models \varphi(b)$ (Definition 4)—this step is where soundness is used and is not available by definition. By Assumptions 1 and 2, $\rightarrow_K^{f,g}$ coincides with $\rightarrow_{\mathrm{EVM}}^{f,g}$ over $\mathcal{E}$, transferring the claim to the real machine. By Assumption 4, b is the deployed artifact, and by Assumption 6 the precondition is satisfiable and reachable, so the quantification is non-empty. Hence every real EVM trace of b from a reachable Pre-configuration satisfies φ in the sense of Definition 2. □

Remark 2 (What the theorem does not claim). The theorem establishes correctness *relative to* φ, not relative to intent: returning (b, φ) does not entail $b \approx n$. The gap between φ and n is closed solely by the human oracle H (Assumption 7), never by the mathematics. This is the single *non-formal* gap; the remaining gaps (model adequacy, checker soundness, artifact identity, environment) are formal and discharged as Assumptions 1–4.

Proposition 1 (Capability-invariance, with non-vacuity caveats). *The conclusion is independent of* F*,* G*, and* I*: replacing any with an arbitrary or adversarial procedure leaves soundness intact; only liveness is affected. Two caveats bound the* usefulness *of the guarantee, not its soundness: an unsatisfiable* Pre *would make the claim vacuous (excluded by Assumption 6), and a too-weak* (Trace, Post, Exit) *would make it true but uninformative. Non-vacuity therefore requires, beyond Assumption 6, external evidence that* φ *is strong enough—e.g. that known-bad contracts are refuted and known-good traces admitted.*

Proof. None of Assumptions 1–7 references F, G, or I; the hypothesis already encodes that Check accepted, so the producer of b, a, c is immaterial to checker soundness. The caveats follow because Definition 2 is satisfied vacuously when Pre is unsatisfiable and trivially when Trace, Post are weak. □

6 Completeness and the Specification Blind Spot

Soundness says an accepted artifact satisfies φ. It says nothing about properties absent from φ.

Corollary 1 (Blind spot). *Let* $\mathrm{closure}(\varphi)$ *be the set of properties entailed by* φ*. For any* $\psi \notin \mathrm{closure}(\varphi)$*, the process makes no claim about the behavior of* b *with respect to* ψ*. In particular, acceptance does not entail the absence of defects outside* φ*.*

The corollary formalizes a single sentence: *silence in the specification is a blind spot, not a guarantee of safety.* The configuration-level specification of Definition 2 widens what *can* be stated—reentrancy and callbacks via Trace, the success/revert/out-of-gas distinction via Exit, emitted logs as trace events—but each must still be written down to be guaranteed. Cross-contract invariants and storage collisions under delegated calls remain outside any single-contract φ unless modeled explicitly.

This relocates the central research problem. The verifier and the machine semantics are strong; the weak link is the *completeness* of φ. The productive role of generative models inside the framework

is therefore not to be trusted with code, but to assist in producing a *more complete* specification—proposing candidate properties and surfacing unspecified paths—while remaining untrusted: every proposed property is validated by H and checked by Check.

Definition 6 (Operational coverage heuristic). Let $\mathcal{P}(b)$ be the set of feasible symbolic execution paths of b under a fixed exploration strategy and let $\mathrm{cov}_\varphi(b)$ be the subset on which φ constrains trace, exit status, or terminal configuration. The ratio $|\mathrm{cov}_\varphi(b)|/|\mathcal{P}(b)|$ is a *heuristic*, not a completeness metric: $\mathcal{P}(b)$ depends on abstraction and solver strategy, a "covered" path may be covered by a trivial constraint, and path coverage is not semantic-property coverage. It is reported as one indicator among several (mutation score, obligation coverage, exit-status coverage, arithmetic-boundary coverage, and an adversarial-property checklist), never as a guarantee of completeness.

7 Discussion

7.1 The role of the invariant synthesizer

Closing a reachability claim over code with loops requires loop invariants, whose discovery is undecidable in general. The Hoare-logic treatment of a loop replaces an unbounded execution by three obligations on a candidate invariant—establishment on entry, preservation by the body, and sufficiency on exit [6]. TCG exploits the resulting asymmetry: although *finding* an invariant is hard, *checking* a proposed invariant reduces to these obligations, each dischargeable by the backend. The synthesizer may therefore guess freely. A wrong invariant fails preservation and is refuted; a too-weak invariant fails the exit obligation and is refuted; only a correct invariant closes the claim, and then by construction. This is the same untrusted-proposal/trusted-check pattern that governs G, applied to annotations in $\mathcal{A}_{\mathrm{chk}}$. It does *not* extend to $\mathcal{A}_{\mathrm{asm}}$, which is why storage-layout facts are handled by Assumption 5 rather than absorbed silently.

7.2 Threats to the trusted base

The assumptions are not free. Adequacy (Assumption 1) is validated by testing, not proof; cross-validation of a critical property against a second, independent EVM semantics [3] is the strongest available mitigation. Checker soundness (Assumption 3) inherits the trust placed in the proof checker and backend; a small, auditable certificate checker is preferable to a monolithic decision procedure precisely because it shrinks this assumption. Artifact identity (Assumption 4) sounds trivial yet is a frequent practical failure when a verified artifact and a deployed artifact differ by metadata or constructor handling.

7.3 Evaluation directions

Soundness non-violation—no accepted pair fails an independent re-check, including against a second semantics—is necessary but not sufficient. A convincing evaluation also exercises the negative space: deliberately buggy contracts, mutated contracts, weakened specifications, fork-mismatch and artifact-mismatch tests, and negative controls for under-specified properties. The conservative property must be checked directly: the system must yield non-acceptance, never a falsely accepted artifact, when the checker returns Unknown.

8 Related Work

The framework sits at the intersection of three lines of work. *Autoformalization* translates informal statements into formal artifacts; its central obstacle—instability under paraphrase [4] and the need to evaluate informal-to-formal alignment explicitly [5], surveyed in [7]—motivates the detect-don't-trust posture adopted here. *Semantics-first verification of EVM bytecode* supplies the sound core and the validated machine semantics on which the soundness theorem rests [1], with independent semantics in other proof systems [3] enabling cross-validation. *Verified compilation* [2] frames both the aspiration and its limit: end-to-end correctness is provable only where both endpoints are formal, which is exactly why the natural-language endpoint must be handled by a human oracle rather than a proof. The framework's distinguishing commitment is to treat the interface itself—the trust boundary—as the object of study.

9 Conclusion

A proof, not a hallucination, is achieved by a hard boundary between untrusted generation and a trusted, formal core: the generator may produce arbitrary output to the left of the core, and the core admits only what its checker can validate against a sound semantics. By separating the operational checker from semantic truth, the soundness theorem is a genuine result rather than a restatement of a definition; by specifying over full configurations, it covers trace, exit-status, and event properties rather than terminal states alone. No bespoke compiler is required; verifying deployed bytecode removes the source compiler from the trusted base. The single *non-formal* gap—the alignment between specification and intent—is discharged as an explicit human-oracle assumption held outside the formal core, and the residual weakness—specification completeness—is named, equipped with an explicitly heuristic coverage proxy, and set out as the principal direction for future work. The core claim is narrow and defensible: *system soundness is invariant to the capabilities of the generative model.*

References

[1] E. Hildenbrandt, M. Saxena, N. Rodrigues, X. Zhu, P. Daian, D. Guth, B. Moore, D. Park, Y. Zhang, A. Stefanescu, and G. Roşu. KEVM: A Complete Formal Semantics of the Ethereum Virtual Machine. In *2018 IEEE 31st Computer Security Foundations Symposium (CSF)*, pp. 204–217, 2018. DOI: 10.1109/CSF.2018.00022.

[2] X. Leroy. A Formally Verified Compiler Back-End. *Journal of Automated Reasoning*, 43(4):363–446, 2009. DOI: 10.1007/s10817-009-9155-4.

[3] F. Cassez, J. Fuller, M. K. Ghale, D. J. Pearce, and H. M. A. Quiles. Formal and Executable Semantics of the Ethereum Virtual Machine in Dafny. In *Formal Methods (FM 2023)*, Lecture Notes in Computer Science, vol. 14000, pp. 571–583. Springer, 2023. DOI: 10.1007/978-3-031-27481-7_32.

[4] H. Moore and A. Shah. Evaluating Autoformalization Robustness via Semantically Similar Paraphrasing. arXiv preprint arXiv:2511.12784, 2025.

[5] J. Lu, Y. Wan, Y. Huang, J. Xiong, Z. Liu, and Z. Guo. FormalAlign: Automated Alignment Evaluation for Autoformalization. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025.

[6] C. A. R. Hoare. An Axiomatic Basis for Computer Programming. *Communications of the ACM*, 12(10):576–580, 1969. DOI: 10.1145/363235.363259.

[7] K. Weng, L. Du, S. Li, W. Lu, H. Sun, H. Liu, and T. Zhang. Autoformalization in the Era of Large Language Models: A Survey. arXiv preprint arXiv:2505.23486, 2025.

[8] Runtime Verification, Inc. KEVM: Semantics of the Ethereum Virtual Machine in the K Framework. Software repository, `https://github.com/runtimeverification/evm-semantics`, accessed 2025.